

DRAFT v1.1

PRODUCTION SCOPE

PYTHON STACK

Product Requirements Document

SecureMailScope — from SIH prototype to a deployable cryptographic posture assessment platform for SOC/forensics/IR teams. Single-language stack: Python end to end.

Applies to	Post-hackathon production build, pilot → departmental → enterprise scale
Architecture	Feature-based (vertical slice) — each feature owns its API, logic and data across layers
Companion doc	SecureMailScope SRS (Prototype) — this PRD extends it, does not replace it

1. Product

1.1 Vision

SecureMailScope becomes the default passive, offline tool a SOC or CERT team reaches for to answer one question: *is our email infrastructure's encryption actually doing its job?* Where existing tools show packets or scan live servers, SecureMailScope reads what already happened and turns it into prioritized, explainable action.

1.2 Personas

Persona	Goal	Success looks like
SOC Analyst (Priya)	Triage large volumes of captured traffic fast	Opens dashboard, sees prioritized Critical list within seconds, not hours of manual packet review
Forensics Investigator (Rakesh)	Build an evidence-grade record for an incident	Exports a signed PDF report with frame-level evidence citations
Mail Infra Admin (Sunita)	Fix what's actually broken, in order	Gets a remediation queue with copy-paste config lines, ranked by real impact

Compliance Officer

Prove posture against a standard

Report maps every finding to NIST SP 800-52r2 / RFC 8996 / RFC 8314

1.3 Goals & success metrics

Goal	Metric	Target (Pilot phase)
Reduce manual triage time	Time from capture upload to prioritized findings	< 5 min for 1 GB capture
Trustworthy AI	% of high-risk verdicts with a visible explanation trace	100%
Catch what rules miss	Anomalies confirmed as real issues / total anomalies flagged	> 60% precision, reviewed quarterly
Operational reliability	Uptime for continuous-ingestion mode	99.5%

2. Architecture Approach

2.1 Why feature-based (vertical slice), not layered

A layered architecture (all routers together, all services together, all data-access together) makes every feature touch every layer — a change to certificate validation requires edits in the routing layer, the service layer, and the data layer, each conceptually a different concern. That slows a team down and makes ownership fuzzy as the system grows past prototype scale.

SecureMailScope's production architecture is organized by **feature** instead: each feature module (§3) owns its own API routes, business logic, data access, and tests, end to end. Modules talk to each other through well-defined interfaces/events, not shared internal state. This lets a feature be independently tested, versioned, scaled and even reimplemented without touching the rest of the system — a genuine engineering benefit, not just organizational tidiness, and it's the same reasoning that let the ML scoring logic sit right next to the rule engine here without either one leaking into the other.

```
src/  
  features/  
    capture_ingestion/      # §3.1 – own router + service + repo + tests  
    protocol_starttls/      # §3.2
```

```

tls_cert_intel/          # §3.3
rule_compliance_engine/ # §3.4
ai_risk_scoring/         # §3.5 – SCoRE fusion + ML calls, same language, same process
reporting/               # §3.6
dashboard_api/           # §3.7
auth_rbac_audit/         # §3.8
shared/
domain_events/           # cross-feature contracts only (pydantic schemas)
common/

```

Single-language advantage: because the whole stack is Python, the AI Risk Scoring feature (§3.5) doesn't need a network hop (gRPC/REST) to reach the ML model — it calls it as a normal in-process function. That removes an entire class of production failure mode (service-to-service auth, network timeout, serialization mismatch) that a polyglot stack would carry. The feature boundary stays clean at the *code* level even though it's not a separate deployable.

3. Feature Modules

Each module below is a self-contained vertical slice: what it owns, its interface to other features, and its production implementation notes.

3.1 Capture Ingestion

Core

Owns	Upload handling, PCAP validation, streaming ingest, TCP stream reassembly
Publishes	StreamReassembled event per completed stream, consumed by 3.2
Story	As an analyst, I upload a multi-GB capture and get an immediate job ID without waiting for processing to finish.
Prod detail	Streaming multipart upload direct to object store via FastAPI's async endpoints; reassembly delegated to tshark subprocess pool per Celery worker; emits one Redis/Kafka event per stream so downstream features scale independently of file size

3.2 Protocol & STARTTLS Analysis

Core

Owns	Protocol classification, STARTTLS state machine, stripping detection
------	--

Consumes	StreamReassembled
Publishes	SessionClassified
Story	As a SOC analyst, I need every session correctly labelled even when servers run mail protocols on non-standard ports.
Prod detail	Banner-based classifier (pure Python), independently unit-tested with pytest against a fixture library of real and synthetic banners

3.3 TLS & Certificate Intelligence

Core

Owns	TLS handshake parsing, X.509 extraction/validation, JA3/JA3S fingerprinting
Consumes	SessionClassified
Publishes	CryptoFactsExtracted
Story	As a forensics investigator, I need certificate chain problems flagged with the exact evidence frame.
Prod detail	Custom byte-level TLS record parser + cryptography (pyca) library for X.509 chain validation; TLS 1.3 encrypted fields explicitly marked not-observable, never silently dropped

3.4 Rule & Compliance Engine

Core

Owns	Deterministic weakness rules, compliance mapping, feature-vector extraction for ML
Consumes	CryptoFactsExtracted
Publishes	FindingsGenerated
Story	As a compliance officer, every finding must cite the exact standard it violates.
Prod detail	Rules externalized as versioned YAML config (not hardcoded), loaded at startup, so the security team can add rules without a code deploy

3.5 AI Risk Scoring — SCoRE

Differentiator

Owns	SCoRE fusion algorithm, scikit-learn classifier/anomaly calls, SHAP explanation trace, prioritization
-------------	---

Consumes	FindingsGenerated
Publishes	SessionScored
Story	As an analyst, I want one trustworthy number per session, with the reasoning visible, not a black box.
Prod detail	SCoRE fusion logic and the ML model both live in this one Python feature module – joblib-loaded model called in-process, no network hop. If model loading or inference fails, this module still returns a complete deterministic-only score (graceful degradation – SRS §4.1) since p_ml simply defaults to 0.5 neutral.

3.6 Reporting

Core

Owns	JSON/PDF/HTML export, report templating, versioned schema
Consumes	SessionScored
Story	As a compliance officer, I export a report today and it still opens correctly in a system built against last year's schema.
Prod detail	Single Jinja2 template rendered to HTML directly and to PDF via WeasyPrint; JSON schema is explicitly versioned (Pydantic model with a schema_version field) for downstream SIEM consumers

3.7 Dashboard & Visualization

Core

Owns	Session table, posture gauge, charts, anomaly panel, drill-down views
Consumes	Dashboard-API (own BFF layer over the above features' read models)
Story	As an analyst, I filter to Critical sessions in one click and drill into evidence without leaving the page.
Prod detail	React SPA calling FastAPI, server-side pagination (TanStack Table) for large session sets, WebSocket push for live-ingestion status

3.8 Auth, RBAC & Audit

Cross-cutting

Owns	Login, role-based access, full audit trail of who accessed which capture/report
Story	As a compliance officer, I can prove exactly who viewed a sensitive finding and when.

Prod detail

FastAPI-Users (or Authlib) for OAuth2/JWT, integrated with the organization's existing directory (LDAP/AD) where available; every capture/report access is an immutable audit log row in PostgreSQL

4. Production Methods

4.1 Production tech stack

Layer	Production choice	Notes
API / core	Python 3.11, FastAPI	Async I/O, auto OpenAPI docs, same language as parsing/ML — zero cross-language friction
PCAP / crypto parsing	dpkt, custom TLS record parser, <code>cryptography</code> (pyca)	Carried straight over from the SRS/prototype implementation — no rewrite needed at production time
Job queue / messaging	Celery + Redis (pilot), Kafka via <code>confluent-kafka-python</code> (departmental+)	Event backbone between feature modules (§3); Redis is enough until continuous multi-site ingestion is needed
ML	scikit-learn, SHAP, joblib	In-process within the AI Risk Scoring feature — no separate service, no gRPC hop
Database	PostgreSQL (primary), TimescaleDB extension for time-series metrics	Feature modules use separate schemas, not separate DBs, for pilot scale
Object storage	MinIO (self-hosted, S3-compatible)	Raw PCAPs, generated reports
Frontend	React + TypeScript, Recharts, TanStack Table	Static build, served independently of backend
PDF generation	Jinja2 + WeasyPrint	Same template drives HTML and PDF — one source of truth
Orchestration	Kubernetes (Stage 2+), Docker Compose (pilot)	HPA on Celery workers by queue depth / consumer lag

4.2 Engineering methods

- **Domain-Driven Design (light):** each feature module in §3 maps to a bounded context; events (Pydantic-typed) are the only cross-context contract
- **API-first:** OpenAPI spec (auto-generated by FastAPI from typed routes) reviewed before implementation for every feature's external interface
- **Contract testing:** event/message schemas are Pydantic models, versioned and validated between producer and consumer features, so one feature's release can't silently break another
- **Trunk-based development** with short-lived feature branches, one branch per vertical slice

4.3 Testing strategy

Level	Scope	Tooling
Unit	Per-feature logic (rule engine, SCoRE formula, parsers)	pytest
Integration	Feature-to-feature via real Redis/Kafka events (Testcontainers)	pytest + Testcontainers-Python
Contract	Event/schema compatibility across features	Pydantic schema validation in CI, versioned schema registry
Security	Static analysis, dependency scanning, fuzzing of the PCAP/TLS parser (malformed input handling)	Bandit, pip-audit / Safety, Atheris (coverage-guided fuzzing) on the parser module
ML evaluation	Precision/recall/F1 per risk class, rule-engine baseline comparison, drift monitoring	scikit-learn metrics, scheduled batch evaluation job (Celery beat)

4.4 DevSecOps practices

- CI/CD per feature module — a change to one feature deploys independently of the others
- Container images scanned pre-deploy (Trivy); no image with known-critical CVEs promoted
- Secrets managed via Vault/K8s secrets, never in code or images
- All inter-feature and external traffic over mTLS in production (the system that assesses TLS posture practices what it preaches)

5. Rollout

5.1 Release phases

Phase	Scope	Exit criteria
Alpha (hackathon)	SRS prototype scope, single node	All 21 PS deliverables demonstrable
Beta / Pilot	One SOC team, real (consented) captures, Docker Compose multi-container	<5 min analysis time on 1 GB capture; zero false-negative on known-weak test corpus
GA / Departmental	Kubernetes, HPA, RBAC, audit trail live	99.5% uptime over 30 days; SIEM JSON integration verified
Enterprise / multi-site	Kafka-buffered continuous ingestion, multi-region workers	Sustained continuous monitoring across >1 site without manual intervention

5.2 Production non-functional requirements

Category	Requirement
Availability	99.5% for continuous-ingestion deployments (Beta+)
Data residency	All processing and storage remain within the deploying organization's network — no external calls, enforced at network-policy level in Kubernetes
Auditability	Every access to a capture or report logged immutably, retained per organizational policy
Explainability	Carried forward unchanged from SRS §7 — every score traceable, always
Scalability	Horizontal scale-out on ingestion and scoring without architecture change (feature modules are independently scalable by design, §2.1)

5.3 Risks & mitigations

Risk	Mitigation
Synthetic training data doesn't generalize to real enterprise traffic	SCoRE degrades gracefully to rule-only scoring; retraining pipeline planned for Beta once consented real captures are available

TLS 1.3 adoption reduces visible certificate data over time	SCoRE explicitly scores on visible metadata + treats TLS 1.3 as a positive signal; SSLKEYLOGFILE support planned for consented lab environments
Kubernetes operational complexity for smaller deployments	Docker Compose remains a fully supported deployment mode through Beta phase, not just a hackathon shortcut
Python performance ceiling on the hottest parsing path (TLS record parsing at very high concurrency)	Identified as the one place to optionally drop to a compiled extension (Cython/Rust via PyO3) later — isolated to one function, doesn't force a language change anywhere else